

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration :** 1 Hour

Total Marks: 50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Eshwanth C 192321062 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Eshwanth C

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Answer:

Problem formulation (state-space search)

- **State:** an ordered pair (x, y) where x = gallons in the 4-gallon jug ($0 \leq x \leq 4$) and y = gallons in the 3-gallon jug ($0 \leq y \leq 3$).
- **Initial state:** $(0, 0)$ – both jugs empty.
- **Goal state:** $(2, n)$ – exactly 2 gallons in the 4-gallon jug; n can be anything $(0-3)$.
- **Actions (production rules):** fill, empty or pour, as listed below. **Path cost:** 1 per action.

Rule	Condition	Action	Result
R1	$x < 4$	Fill the 4-gallon jug from the pump	$(4, y)$
R2	$y < 3$	Fill the 3-gallon jug from the pump	$(x, 3)$
R3	$x > 0$	Empty the 4-gallon jug on the ground	$(0, y)$
R4	$y > 0$	Empty the 3-gallon jug on the ground	$(x, 0)$
R5	$x + y \geq 4$ and $y > 0$	Pour from 3-gal jug into 4-gal jug until it is full	$(4, y - (4 - x))$
R6	$x + y \geq 3$ and $x > 0$	Pour from 4-gal jug into 3-gal jug until it is full	$(x - (3 - y), 3)$
R7	$x + y \leq 4$ and $y > 0$	Pour all the water from 3-gal jug into 4-gal jug	$(x + y, 0)$
R8	$x + y \leq 3$ and $x > 0$	Pour all the water from 4-gal jug into 3-gal jug	$(0, x + y)$

Solution (found by breadth-first search, so it is a shortest solution – 6 moves)

Step	4-gal jug (x)	3-gal jug (y)	State	Rule applied
0	0	0	$(0, 0)$	Start
1	0	3	$(0, 3)$	R2 – fill the 3-gallon jug
2	3	0	$(3, 0)$	R7 – pour all of 3-gal jug into 4-gal jug
3	3	3	$(3, 3)$	R2 – fill the 3-gallon jug again
4	4	2	$(4, 2)$	R5 – pour 3-gal into 4-gal until the 4-gal is full (only 1 gallon fits)
5	0	2	$(0, 2)$	R3 – empty the 4-gallon jug
6	2	0	$(2, 0)$	R7 – pour the 2 gallons from the 3-gal jug into the 4-gal jug GOAL

State path: $(0,0) \rightarrow (0,3) \rightarrow (3,0) \rightarrow (3,3) \rightarrow (4,2) \rightarrow (0,2) \rightarrow (2,0)$

Alternative 6-move solution: $(0,0) \rightarrow (4,0) \rightarrow (1,3) \rightarrow (1,0) \rightarrow (0,1) \rightarrow (4,1) \rightarrow (2,3)$ (fill 4, pour into 3, empty 3, pour 1 into 3, fill 4, top up the 3-gal jug – leaving exactly 2 gallons in the 4-gallon jug).

Result: Exactly 2 gallons can be measured in the 4-gallon jug in 6 operations. The 3-gallon jug acts as a measuring device: $3 + 3 - 4 = 2$ gallons.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

- i. What are the percept's for this agent?
- ii. Characterize the operating environment.
- iii. What are the actions the agent can take?
- iv. How can one evaluate the performance of the agent?
- v. What sort of agent architecture do you think is most suitable for this agent? Why?

Answer:

PEAS description of the Mars Rover (a robotic scientist-agent)

PEAS	Mars Rover
Performance measure	Number and scientific value of samples analysed; amount of useful data returned to Earth; area/distance explored safely; rover safety (no tip-over, no getting stuck, no damage); energy efficiency; mission lifetime.
Environment	Martian surface: rocks, soil, sand, craters, slopes, dust storms, low temperature, thin atmosphere, day/night cycle; Earth control centre and orbiters as communication partners.
Actuators	Wheels/motors, steering, robotic arm, drill/scoop/brush, camera mast, antenna, solar-panel/heater controls.
Sensors	Cameras (stereo/panoramic), spectrometers, LIDAR/proximity, inertial unit (tilt/slope), wheel odometry, temperature, radiation and dust sensors, battery/power monitors, antenna.

i. Percepts

- Camera images of terrain, rocks and horizon (panoramic and stereo).
- Spectrometer / chemical-analysis readings of soil and rock samples (e.g., mineral composition, presence of water or organics).
- Terrain slope, tilt and acceleration from the inertial measurement unit; wheel odometry and wheel-slip/motor-current readings.
- Obstacle / distance data from LIDAR or proximity sensors; drill and arm torque feedback.
- Temperature, radiation, wind/dust level, sun position and time of day.
- Battery level and solar-panel power; internal health/fault status.
- Commands and updates received from Earth or from the orbiter.

ii. Operating environment (properties)

Property	Type	Reason
Observable	Partially observable	Sensors see only nearby terrain; far-away hazards and sample contents are hidden.
Agents	Single agent (cooperating with Earth/orbiters)	No competing agents on Mars.
Deterministic?	Stochastic	Wheel slip, sliding sand, sensor noise, dust storms make outcomes uncertain.
Episodic?	Sequential	Current moves affect energy, position and later opportunities.
Static?	Dynamic	Weather, light and temperature change while the rover is deliberating.
Discrete?	Continuous	Position, speed, temperature and time are continuous.
Known?	Partly unknown	Terrain and samples are new, so the rover must learn; 4–24 min communication delay prevents real-time remote driving.

iii. Actions

- Move forward / backward, turn left / right, stop, climb, and change speed; plan and follow a path; avoid obstacles and slopes.
- Position the camera mast and take images; deploy the robotic arm; drill, scoop, brush or scrape a sample.
- Place the sample in an instrument, run the analysis, store or discard the sample.
- Transmit data to Earth/orbiter and receive commands; schedule transmissions.
- Adjust solar panels, recharge, switch on heaters, enter sleep/safe mode; run self-diagnosis.

iv. Performance evaluation

- **Science return:** number of valuable samples analysed, accuracy of analyses, quantity and quality of data sent to Earth.
- **Exploration:** area/distance covered and number of target sites reached per unit time.
- **Safety:** number of collisions, tip-overs, stuck events or hardware damage (should be zero).
- **Efficiency:** energy used per task, time per sample, communication success rate.
- **Longevity:** mission duration and uptime; overall utility = weighted sum of the above (science value ↑, risk and energy ↓).

v. Most suitable agent architecture

A **goal-based agent with a model-based reflex layer and a utility/learning component** (a hybrid, layered architecture) is the most suitable:

- **Model-based:** the environment is partially observable, so the rover must keep an internal map/state (position, battery, terrain, sample status).
- **Reflex layer:** safety needs instant reactions (stop if tilt is too high, avoid a cliff edge, go to safe mode when power is low).
- **Goal-based:** missions have explicit goals (“reach rock X, drill, analyse”), which need planning and search over sequences of actions; because of the communication delay, the rover must plan on its own.
- **Utility-based:** it must trade off science value against risk and energy when several targets are possible.
- **Learning:** improves terrain classification, traversability estimates and sample selection from experience.

A simple reflex agent is not enough because it has no memory and cannot plan, and a purely goal-based agent cannot react quickly enough to hazards.

```
Sensors --> [ Percept processing / world model ] --> [ Goal & utility planner ] --> Actuators
              |                                     ^
              +---> [ Reflex safety layer (condition-action rules) ] ---+
                    (Learning element updates the world model and the utility function)
```

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Answer:

Problem formulation (incremental formulation)

Component	Definition
States	Any arrangement of 0 to 8 queens on the board. (Improved version: n queens placed in the n leftmost columns, one per column, with none attacking another.)
Initial state	Empty board – no queens placed.
Actions	Add a queen to the leftmost empty column, in a row where it is not attacked by any queen already on the board.
Transition model	Returns the board with the new queen added to the chosen square.
Goal test	8 queens are on the board and none is attacked (no two share a row, column or diagonal).
Path cost	Not relevant – only the final configuration matters (every action costs 1).

Why the figure is not a solution

Two queens attack each other if they share a row, a column, or a diagonal. In the given configuration, the queen in the first column and the queen in the last column lie on the same diagonal, so the goal test fails. A complete-state formulation would measure this with the heuristic h = number of pairs of queens attacking each other ($h = 0$ only at a solution).

Size of the search space

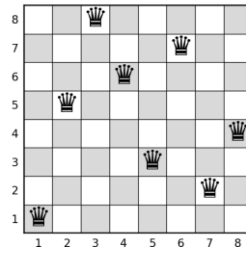
Formulation	Number of states / sequences
Place 8 queens on any squares (ordered sequences) $64 \times 63 \times \dots \times 57$	$\approx 1.8 \times 10^{14}$
Choose any 8 of 64 squares: $C(64, 8)$	$4,426,165,368 \approx 4.4 \times 10^9$
One queen per column, any row: 8^8	16,777,216
One queen per row and column (permutations): $8!$	40,320
Incremental, non-attacking only (improved)	2,057

Exploring the search space with search strategies

- **Depth-first search with backtracking (best uninformed choice):** all solutions are at depth 8, so DFS needs only $O(bm)$ memory. It places a queen column by column, and when a column has no safe row it backtracks to the previous column. With rows tried in order 1...8, the first solution found is rows (1, 5, 8, 6, 3, 7, 2, 4) for columns 1–8.
- **Breadth-first search:** complete and finds a goal at depth 8, but must store all partial boards at each level (up to a few hundred in the improved formulation), so memory is far greater than DFS; no benefit since path cost is irrelevant.
- **Depth-limited search (limit = 8):** same as DFS, since no path is longer than 8.
- **Local search (hill climbing / min-conflicts) with complete-state formulation:** start with 8 queens (one per column) and move one queen within its column to reduce h ; very fast for large n but can get stuck in local minima and needs random restarts.

The problem has 92 solutions in total, 12 of which are distinct after rotations and reflections.

A valid solution (no queen attacks another)



Solution above: queen rows by column = (1, 5, 8, 6, 3, 7, 2, 4); no two share a row, column or diagonal.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Answer:

Type of agent

A **goal-based problem-solving agent** is suitable: it has a clear goal (reach the destination), formulates the problem, searches for a sequence of actions (road segments) that reaches the goal, and then executes the sequence. Because the customer can choose among several cab types (mini, micro, sedan, shared, prime ...) based on comfort, fare and time, the agent also uses a **utility function** to compare the options (a goal-based agent with a utility-based selection step).

Problem formulation

Component	OLA cab booking
Initial state	Customer's pickup location (current GPS position).
Goal state	Customer's drop location (destination).
Actions	Select cab type; drive along a road segment from one junction to the next; (accept ride / start trip / end trip).
Transition model	Moving along a road moves the cab from junction i to junction j.
Path cost	Distance, travel time and fare (depends on the cab type, traffic, surge); Utility = comfort – fare – waiting/travel time.
Environment	Road map (graph), traffic, available cabs, user preferences – a partially observable, dynamic, stochastic environment.

Pseudocode

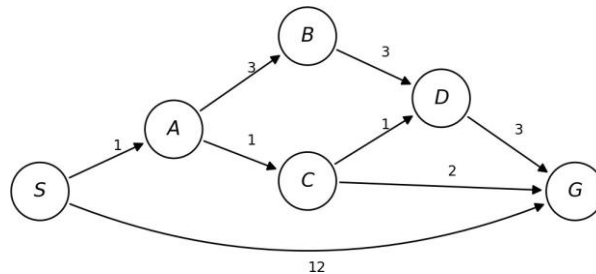
```
function OLA-CAB-AGENT(percept) returns an action
  persistent: seq      // action sequence, initially empty
                state   // current description of world (location, traffic, cabs)
                goal     // initially null
                problem  // problem formulation
                prefs    // customer's budget, comfort, number of passengers

  state <- UPDATE-STATE(state, percept)      // pickup, drop, traffic, cab availability
  if seq is empty then
    goal <- FORMULATE-GOAL(state)           // goal: be at destination
    cabs <- GET-AVAILABLE-CABS(state)       // {mini, micro, sedan, shared, prime, ...}
    bestU <- -infinity; bestCab <- null; bestSeq <- failure
    for each cab in cabs do
      problem <- FORMULATE-PROBLEM(state, goal, cab)
      route <- SEARCH(problem)              // A*/UCS on road graph (cost = time/distance)
      if route != failure then
        fare <- ESTIMATE-FARE(route, cab)
        eta <- ESTIMATE-TIME(route, cab)
        U <- w1*COMFORT(cab) - w2*fare - w3*eta // utility
        if U > bestU then bestU <- U; bestCab <- cab; bestSeq <- route
    if bestSeq = failure then return NO-CAB-AVAILABLE
    CONFIRM-BOOKING(bestCab)                // user confirms; driver assigned
    seq <- bestSeq
  action <- FIRST(seq)                      // e.g. drive to next junction
  seq <- REST(seq)
  return action

function SEARCH(problem) returns a route or failure
  frontier <- priority queue with initial state(pickup), ordered by path cost
  explored <- empty set
  loop do
    if frontier is empty then return failure
    node <- POP(frontier)
    if GOAL-TEST(node.state) then return SOLUTION(node) // destination reached
    add node.state to explored
    for each action in ACTIONS(node.state) do // next road segments
      child <- CHILD-NODE(problem, node, action)
      if child.state not in explored and not in frontier then INSERT(child, frontier)
      else if child.state in frontier with higher cost then replace it with child
```

The agent re-plans (calls the function again) if the percept shows a change such as heavy traffic, a road block or driver cancellation.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm. The delivery network is represented as a weighted graph:



Answer:

Problem understanding

Directed weighted graph with edges: $S \rightarrow A = 1$, $S \rightarrow G = 12$, $A \rightarrow B = 3$, $A \rightarrow C = 1$, $B \rightarrow D = 3$, $C \rightarrow D = 1$, $C \rightarrow G = 2$, $D \rightarrow G = 3$. Start = S, Goal = G. Edge costs are in cost units (fuel + time).

Method: Uniform Cost Search

UCS always expands the frontier node with the lowest path cost $g(n)$ (priority queue). The goal test is applied when a node is **selected for expansion**, not when it is generated, so the first time G is popped its cost is the minimum. UCS is complete (step costs > 0) and optimal.

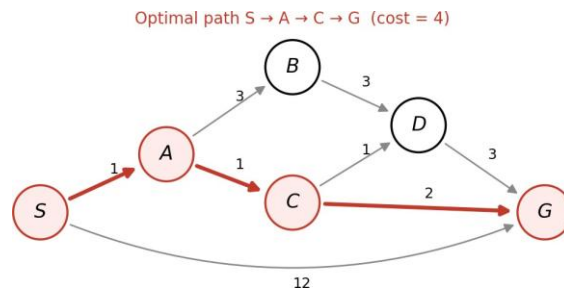
Solution process

Step	Node expanded (g)	Nodes generated / updated	Frontier (priority queue) after step	Explored
0	–	–	S(0)	{ }
1	S (0)	A = 0+1 = 1; G = 0+12 = 12	A(1), G(12)	{S}
2	A (1)	B = 1+3 = 4; C = 1+1 = 2	C(2), B(4), G(12)	{S, A}
3	C (2)	D = 2+1 = 3; G = 2+2 = 4 (better than 12, so G is updated)	D(3), B(4), G(4)	{S, A, C}
4	D (3)	G via D = 3+3 = 6 $>$ 4 $\rightarrow$ ignored	B(4), G(4)	{S, A, C, D}
5	B (4)	D via B = 4+3 = 7 $\rightarrow$ D already explored, ignored	G(4)	{S, A, C, D, B}
6	G (4)	Goal test passed – STOP	–	{S, A, C, D, B, G}

Order of expansion: $S \rightarrow A \rightarrow C \rightarrow D \rightarrow B \rightarrow G$. **Path (using parent pointers $G \leftarrow C \leftarrow A \leftarrow S$):** $S \rightarrow A \rightarrow C \rightarrow G$.

Comparison of all possible paths

Path from S to G	Total cost
$S \rightarrow G$	12
$S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$	$1 + 3 + 3 + 3 = 10$
$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$	$1 + 1 + 1 + 3 = 6$
$S \rightarrow A \rightarrow C \rightarrow G$	$1 + 1 + 2 = 4$ (minimum)



Interpretation of result

- The least-cost route from warehouse S to warehouse G is $S \rightarrow A \rightarrow C \rightarrow G$ with a total cost of **4 units** (fuel + time).
- The direct route $S \rightarrow G$ costs 12 units, so the optimal route saves 8 units (about 67%) even though it passes through more warehouses.
- UCS expanded 5 nodes (S, A, C, D, B) before selecting G; only B and D needed to be explored because their costs were not larger than the optimal cost 4.
- UCS is optimal because it expands nodes in order of increasing path cost. Time and space complexity: $O(b^{1 + C^*/\epsilon})$, where $C^* = 4$ is the optimal cost and ϵ the smallest edge cost (1).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem